

Encoding:

1. Input Encoding: Translating Conditions into Geometry

When the model receives a condition like Sunday, December, it cannot simply ingest raw integers like 6 for Sunday and 11 for December. If it did, the neural network might falsely assume that Sunday is mathematically **six times larger** than Monday, which is misleading.

To solve this, the input parameters are converted into dense geometric vectors using two techniques:

- **Categorical Embeddings:** Discrete variables like the "Decade" or "Leap Year Status" are mapped to unique, high dimensional coordinate vectors that the model can adjust during training.
- **Cyclical Encoding:** Standard neural networks do not understand that time is a circle. For example: December wraps directly back around to January. To force the model to understand the circular nature of the calendar, periodic variables like the Day of the Week and Month are transformed using trigonometry.

Equation: For a given Day of the Week, d , where d in $[0,1,..,6]$, the cyclical token representation is defined as:

$$x_{cyclical} = \left[\sin \left(\frac{2\pi d}{7} \right), \cos \left(\frac{2\pi d}{7} \right) \right]$$

Example:

Using this equation, Monday ($d=0$) is encoded as the vector $[0.0, 1.0]$. Sunday ($d=6$) is encoded as $[-0.78, 0.62]$. When plotted on a circle, these two vectors sit perfectly adjacent to one another. This **mathematically prevents the model from being confused by the numerical jump** from 6 back to 0 at the end of the week.

2. Output Encoding: Target Probability Distributions

During training, the model must learn to predict the correct date based on the input conditions. The **true answer** is rarely a single number. If the condition is to generate a **Friday in December 1960**, there are multiple mathematically correct answers.

Therefore, the output tokenizer treats date generation as a classification problem. It maps the target date into a **Multi Hot Probability Distribution**. It creates an array representing all possible choices (**31 days**) and distributes **100%** of the probability mass equally among all valid, mathematically correct days, assigning **0%** to everything else.

Equation: Let V be the total number of valid days that satisfy the given calendar conditions, and i be a specific day of the month $1 \leq i \leq 31$. The target probability, $P(i)$, for predicting day i is defined as:

$$P(i) = \begin{cases} \frac{1}{V} & \text{if day } i \text{ is a valid match} \\ 0 & \text{otherwise} \end{cases}$$

Example:

Assume the model is predicting the "Day" for a Friday in December 1960. A quick look at a calendar shows there are exactly 5 Fridays in that month (the 2nd, 9th, 16th, 23rd, and 30th).

Therefore, $V = 5$. The "tokenized" target becomes an array of 31 elements where the correct days are assigned a probability of 0.20 (20% chance each):

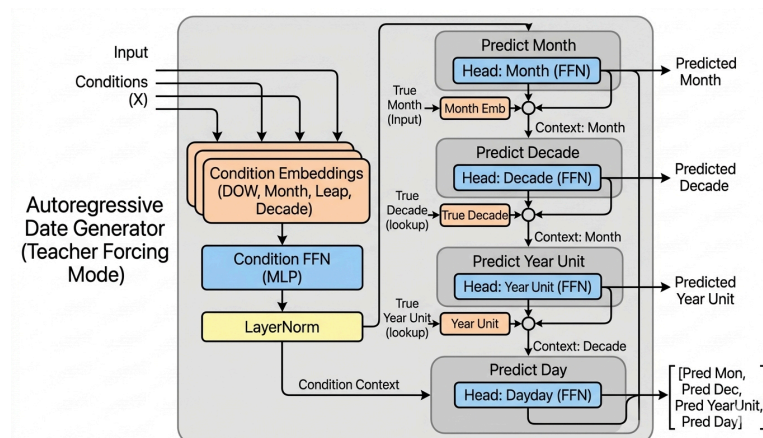
- Target for Day 2 = 0.20
- Target for Day 9 = 0.20
- Target for Day 10 = 0.00 (Incorrect day)
- Target for Day 31 = 0.00 (Incorrect day)

By encoding the targets as dynamic probability distributions, the model is penalized if it guesses impossible dates, **but equally rewarded for picking any of the correct mathematical options**. This ensures the generator maintains high diversity without sacrificing strict accuracy.

Model 1: Autoregressive Classifier

A single neural network with a shared ConditionEncoder and four independent prediction heads, one per AR step. Training uses KL divergence against multi-hot target distributions (uniform over all valid choices at each step), so the model learns to place probability mass on any valid date rather than a single ground truth. Sampling follows the AR order with hard masking of invalid logits.

- **Loss:** sum of four KL divergences (one per AR head)
- **Sampling:** greedy month/decade, multinomial over leap-valid year units, multinomial over DOW-valid days
- **Config:** Epochs: 100, lr: $2e-3$, batch: 256, cosine LR schedule

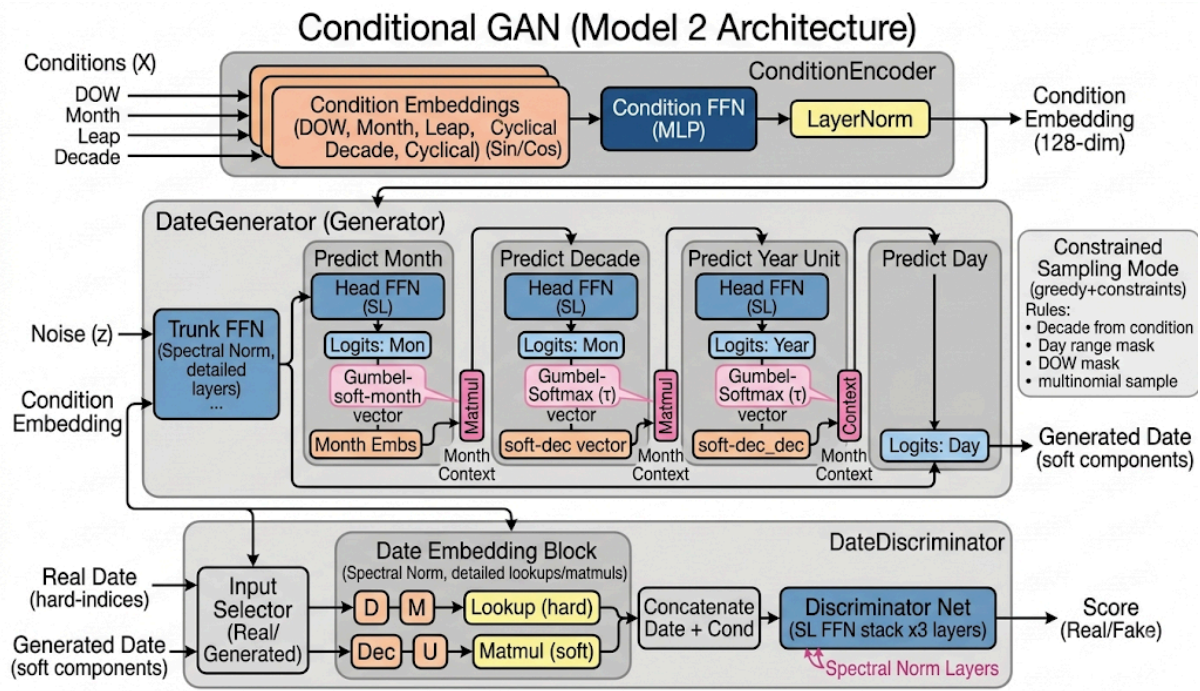


Model 2: Conditional GAN (WGAN-GP + Auxiliary KL)

A GAN in which the Generator maps noise $z \sim N(0, I) + \text{condition embedding}$ to date logits via the same AR head structure. The Discriminator uses spectral normalisation for WGAN-GP stability. The Generator

has an additional auxiliary KL supervision term (same targets as Model 1) to prevent mode collapse and keep heads semantically grounded.

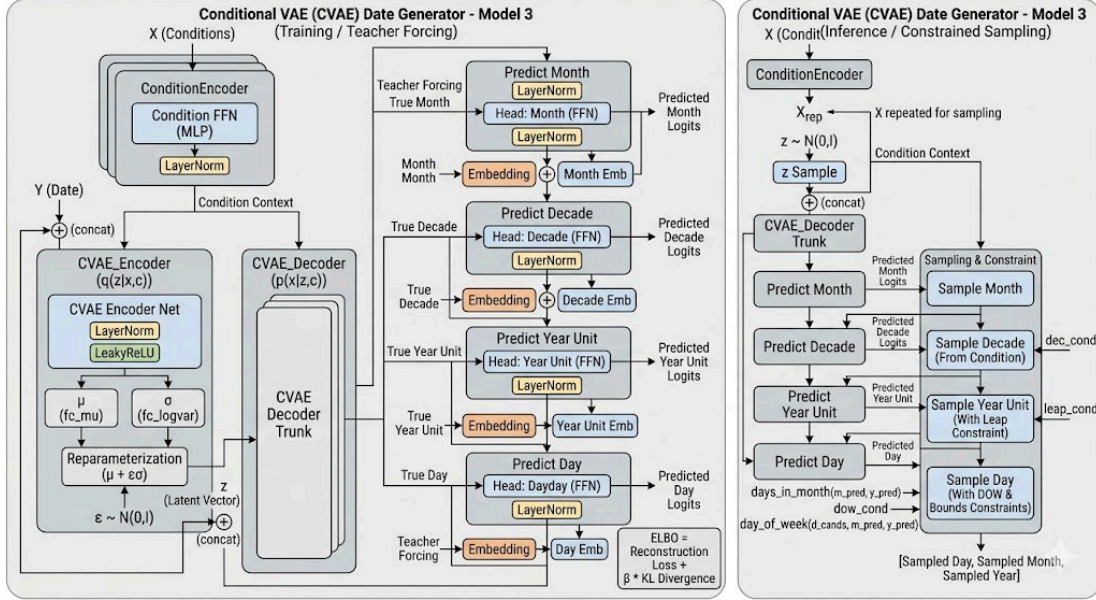
- **Generator:** trunk MLP $\rightarrow$ 4 AR heads; Gumbel-softmax (τ annealed 2.0 $\rightarrow$ 0.5) for differentiable discrete samples
- **Discriminator:** embeds the date components and the condition, then scores via a spectral-norm MLP
- **Loss:** WGAN-GP (computed in embedding space) + $\lambda_{\text{aux}}=0.5 \times \text{KL}$
- Encoder warm started from Model 1 weights and frozen during GAN training
- **Config:** Epochs: 200, lr_G: 1e-4, lr_D: 4e-4, n_critic: 2, λ_{gp} : 10



Model 3: Conditional VAE (CVAE)

A standard CVAE: a recognition network $q(z|\text{date}, \text{cond})$ encodes a real date + condition into a Gaussian posterior; a decoder $p(\text{date}|z, \text{cond})$ uses the same 4-head AR structure. Training maximises the ELBO with β -annealed KL (linear warm-up over the first 50% of epochs to prioritise reconstruction quality early on).

- **Encoder:** date embeddings + condition $\rightarrow (\mu, \log\text{var}) \in \mathbb{R}^{64}$.
- **Decoder:** z + condition $\rightarrow$ 4 AR logit heads (teacher-forced); cross-entropy reconstruction loss.
- **Loss:** ELBO = Recon + $\beta \times \text{KL}$ (β annealed 0 $\rightarrow$ 0.5).
- **Inference:** sample $z \sim \mathcal{N}(0, I)$, decode with constrained AR sampling.
- **Config:** Epochs: 200, lr: 1e-3.



Model 4: Conditional Diffusion Model

A conditional denoising diffusion probabilistic model is trained to generate valid calendar dates from a four component condition vector (day of week, month, leap year flag, decade index). The forward process corrupts a learned continuous date embedding with Gaussian noise over $T=500$ timesteps using a linear beta schedule ($\beta_1=1 \times 10^{-4}$ to $\beta_T=0.02$). A denoising network learns to predict the clean embedding x_0 directly (predict- x_0 objective) conditioned on both the timestep and the condition vector via FiLM (Feature-wise Linear Modulation) residual blocks. Auxiliary discrete cross-entropy heads on month, decade, year-unit, and day provide a strong gradient signal that guides the network to learn calendar structure in parallel with the MSE objective. Inference uses DDIM (50 steps), reducing sampling time by $\sim 10\times$ compared to full DDPM while remaining deterministic. The final output is projected onto a valid calendar date by applying constraint masks (month fixed by condition, decade fixed by condition, leap-year constraint on year unit, day-of-week constraint on day) identically to Models 1–3 (Ho et al., NeurIPS 2020; Song et al., ICLR 2021).

- **Architecture:** ConditionEncoder + SinusoidalTimeEmbedding (128 dims) + DateEmbedder (4×32 dims) + DenoisingNet (6 FiLM residual blocks, 512 hidden)
- **Loss:** $L = \text{MSE}(x_0_{\text{pred}}, x_0_{\text{embed}}) + 0.1 \times \text{CE}(\text{discrete heads}, Y)$
- **Diffusion schedule:** $T=500$ timesteps, linear β schedule, predict- x_0 parameterisation
- **Inference:** DDIM, 50 steps (deterministic); constrained projection enforces all four calendar conditions at decode time
- **Config:** Epochs: 100, lr: 1×10^{-4} , batch size: 256, gradient clipping: 1.0

